

# Test Scenarios Guide

Below is a **practical mapping of R1–R25 onto the 10 mandatory tests**, plus **what each test should actually do** (stimulus + checks + coverage targets). This is written so your testbench team can implement directly.

## 1) Requirement → Test mapping (traceability)

Legend: **P** = primary target of that test, **S** = secondary (covered incidentally / partially)

| Req | What it means (short)                                           | Main tests that must cover it                                                  |
|-----|-----------------------------------------------------------------|--------------------------------------------------------------------------------|
| R1  | R/W regs read back last written (mask RO)                       | <b>reg_access_test (P)</b> , sanity_test (S), interrupt_test (S)               |
| R2  | Reset values correct                                            | <b>reg_access_test (P)</b> , sanity_test (S)                                   |
| R3  | EN=0 holds idle/resets FIFOs+shifter; SS forced high; SCLK idle | <b>sanity_test (S)</b> , <b>error_injection_test (P)</b> , reg_access_test (S) |
| R4  | SCLK idle polarity matches CPOL                                 | <b>mode_coverage_test (P)</b> , sanity_test (S)                                |
| R5  | MOSI stable around sample edge, changes on launch edge per mode | <b>mode_coverage_test (P)</b>                                                  |
| R6  | MSB/LSB shifting order TX/RX                                    | <b>mode_coverage_test (P)</b> , width_coverage_test (S)                        |
| R7  | Transfer lasts WIDTH cycles; BUSY timing                        | <b>mode_coverage_test (P)</b> , width_coverage_test (P), fifo_stress_test (S)  |
| R8  | SCLK freq = PCLK/(2*(DIV+1))                                    | <b>clk_div_corner_test (P)</b>                                                 |
| R9  | TX writes accepted when not full, FIFO order                    | <b>fifo_stress_test (P)</b> , sanity_test (S)                                  |
| R10 | RX reads pop FIFO order                                         | <b>fifo_stress_test (P)</b>                                                    |
| R11 | TX depth=8; TX_FULL on 8th                                      | <b>fifo_stress_test (P)</b> , error_injection_test (S)                         |
| R12 | RX depth=8; RX_FULL on 8th                                      | <b>fifo_stress_test (P)</b>                                                    |
| R13 | TX write while full discarded + TX_OVF set                      | <b>error_injection_test (P)</b> , fifo_stress_test (S), interrupt_test (S)     |
| R14 | RX push while full discarded + RX_OVF set                       | <b>error_injection_test (P)</b> , fifo_stress_test (S), interrupt_test (S)     |
| R15 | RX read while empty returns 0, no RX_OVF                        | <b>error_injection_test (P)</b> , fifo_stress_test (S)                         |

| Req | What it means (short)                                           | Main tests that must cover it                                                                                                     |
|-----|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| R16 | IRQ == OR(INT_STAT & INT_EN) always                             | <b>interrupt_test (P)</b> (plus SVA)                                                                                              |
| R17 | INT_STAT W1C behavior                                           | <b>interrupt_test (P)</b>                                                                                                         |
| R18 | W1C race: event same cycle as clear => stays 1                  | <b>interrupt_test (P)</b>                                                                                                         |
| R19 | Loopback: MOSI internally to RX, ignore external MISO           | <b>loopback_test (P)</b>                                                                                                          |
| R20 | SS_n combinational from SS_CTRL; IP never toggles               | <b>sanity_test (S)</b> , <b>ss control part of fifo_stress/mode tests</b> , <b>error_injection_test (S)</b>                       |
| R21 | DELAY inserts idle half-cycles between words while BUSY stays 1 | <b>delay_transfer_test (P)</b>                                                                                                    |
| R22 | APB: PREADY=1 always, PSLVERR=0 always                          | <b>reg_access_test (P)</b> , apb protocol checks inside all tests (S)                                                             |
| R23 | Reserved offsets read 0, writes ignored                         | <b>error_injection_test (P)</b> , <b>reg_access_test (S)</b>                                                                      |
| R24 | DIV=0 => SCLK=PCLK/2                                            | <b>clk_div_corner_test (P)</b>                                                                                                    |
| R25 | DIV/MODE/WIDTH/LSB sampled at transfer start, held for transfer | <b>clk_div_corner_test (S)</b> , <b>mode_coverage_test (S)</b> , <b>width_coverage_test (P)</b> , <b>error_injection_test (S)</b> |

**Conclusion:** All R1–R25 can be covered with the 10 mandatory tests if each test is written “deeply” (not just a shallow demo).

## 2) What each mandatory test should do (beginner-friendly recipe)

For each test below:

- **Stimulus** = what to drive
- **Checks** = what to verify (scoreboard + direct reads + SVA)
- **Coverage hooks** = what bins it should hit

### 1) **sanity\_test**

**Purpose:** prove your TB can configure DUT and complete 1 clean transfer.

#### Stimulus (minimum)

1. Apply reset, release it.
2. Program:
  - `CTRL` : EN=1, MSTR=1, MODE=0, WIDTH=8, LSB\_FIRST=0, LOOPBACK=0.
  - `CLK_DIV` = small (e.g., 0 or 1).
  - `SS_CTRL` : enable SS0 and drive low (assert).
3. Write 1 byte to `TX_DATA` .
4. SPI slave returns a known byte on MISO (e.g., 8'hA5).
5. Wait for `STATUS.BUSY` to go 1 then back to 0, and TX empty.
6. Read `RX_DATA` and compare to expected response.
7. Deassert SS.

## Checks (targets)

- Basic correct SPI shifting and RX capture.
- `STATUS.BUSY` behavior looks sane.
- SS is held low during transfer (can be SVA monitored).
- No interrupts necessarily required here.

## Covers mainly

- Partial: R3, R4 (mode0 idle low), R7, R9/R10 (basic), R20.

---

## 2) `reg_access_test`

**Purpose:** full register file correctness + reset behavior + APB constants.

## Stimulus

1. After reset: read all registers and compare to reset values.
2. For each RW register: write a pattern then read back and compare (mask RO bits).
  - `CTRL`, `CLK_DIV`, `SS_CTRL`, `INT_EN`, `DELAY`
3. For RO registers: ensure writes don't change them (`STATUS`).
4. For special regs:
  - `TX_DATA` : read returns 0, write pushes data (do not require shift to occur here).
  - `RX_DATA` : write ignored; read pops if data exists.

## Checks

- Strict readback (R1).

- Reset values (R2).
- APB: `PREADY==1` , `PSLVERR==0` always (direct sample or assertion).

## Covers mainly

- R1, R2, R22 (and parts of R23 depending if you include reserved offsets).
- 

### 3) `mode_coverage_test`

**Purpose:** verify all 4 SPI modes × both bit orders × widths {8,16,32}.

## Stimulus

Loop over:

- MODE = 0..3
- LSB\_FIRST = 0/1
- WIDTH = 8/16/32

For each combination:

1. Configure CTRL accordingly.
2. Configure SS asserted.
3. Drive one TX word with recognizable bit pattern (not symmetric).
4. SPI slave returns known word (different per combo).
5. Wait transfer done, read RX, compare.
6. Deassert SS.

## Checks

- SCLK idle polarity matches CPOL when not busy (SVA).
- MOSI stable around sample edges, changes on launch edges (SVA + monitor).
- Correct bit order (LSB/MSB) on observed MOSI and received RX data.

## Covers mainly

- R4, R5, R6, R7 (strong).
- 

### 4) `width_coverage_test`

**Purpose:** width boundary correctness and “exactly WIDTH cycles” correctness.

## Stimulus ideas

1. Run 8-bit transfers with values that stress MSB/LSB (e.g., 8'h01, 8'h80, 8'hFF).
2. Run 16-bit transfers with patterns like 16'h0001, 16'h8000, 16'hA55A.
3. Run 32-bit transfers with 32'h0000\_0001, 32'h8000\_0000, 32'hDEAD\_BEEF.
4. For each:
  - measure number of SCLK toggles / sample edges during transfer
  - ensure BUSY deassert timing matches spec
5. (Optional but very useful for R25): attempt to write CTRL.WIDTH during BUSY=1 and show it does not affect the current transfer (it's illegal sequence, but R25 says sampled at start).

## Checks

- Transfer length matches width.
- BUSY timing and end-of-transfer.
- No truncation/extension.

## Covers mainly

- R7, R25, and supports R6.

---

## 5) `fifo_stress_test`

**Purpose:** FIFO depth, ordering, back-to-back transfers, full/empty flags.

## Stimulus

1. Configure a stable mode/width (pick mode0, width=8 or 16 initially).
2. Assert SS.
3. Push a burst:
  - fill TX FIFO with 8 entries while EN=1
  - confirm `TX_FULL` asserted on 8th, `TX_EMPTY` deasserts
4. Start transfers (clock runs when TX non-empty + SS asserted + EN=1).
5. Read RX gradually:
  - check `RX_EMPTY` changes correctly
  - read 8 entries and confirm order matches slave responses
6. Repeat with back-to-back transfers (no gaps), and include a case where RX becomes full.

## Checks

- TX order preserved (R9).
- RX pop order preserved (R10).
- TX depth exactly 8 (R11).
- RX depth exactly 8 (R12).
- Status flags update correctly (part of many bugs).

## Covers mainly

- R9–R12 strongly, and sets up overflow cases for R13/R14 optionally.
- 

## 6) `interrupt_test`

**Purpose:** all 5 interrupts + mask behavior + W1C + race.

Interrupt bits:

- `TRANSFER_DONE`, `RX_OVF`, `TX_OVF`, `RX_FULL`, `TX_EMPTY`

## Stimulus plan (structured)

For each interrupt source:

1. Clear `INT_STAT` (W1C all bits).
2. Enable only that interrupt in `INT_EN`.
3. Cause the event.
4. Check:
  - `INT_STAT` bit set (sticky)
  - IRQ asserted iff enabled
5. Mask it (`INT_EN` bit = 0), cause event again:
  - `INT_STAT` still sets
  - IRQ remains low

### W1C check

- Write 1 to clear that bit and confirm it clears.

### Race check (R18)

- Try to schedule a W1C write in the same PCLK cycle the event occurs.

- Expectation: bit remains 1 (event wins).

## Checks

- `IRQ == |(INT_STAT & INT_EN)` continuously (best as SVA + runtime sampling).
- W1C correctness and race.

## Covers mainly

- R16, R17, R18 (and overflow-related R13/R14 support).
- 

## 7) `clk_div_corner_test`

**Purpose:** verify SCLK frequency equation and DIV corner cases, incl DIV=0.

## Stimulus

For DIV in {0,1,2,3,255,1024,65535} (at least the contract subset: 0,1, small, >=1024):

1. Configure mode known (mode0) and width constant.
2. Start a transfer.
3. Measure SCLK half-period (in PCLK cycles) from toggles:
  - expected half-period = (DIV+1) PCLK cycles
  - expected SCLK full period = 2\*(DIV+1) PCLK cycles
4. Confirm DIV=0 gives SCLK = PCLK/2.

## Checks

- Numeric tolerance: because everything synchronous to PCLK, you can check exact cycle counts.
- Also a good place to test R25: write CLK\_DIV mid-transfer and ensure current transfer timing doesn't change.

## Covers mainly

- R8, R24, and part of R25.
- 

## 8) `loopback_test`

**Purpose:** loopback overrides external MISO.

## Stimulus

1. Set CTRL.LOOPBACK = 1.
2. Drive external MISO with garbage pattern (even random every edge).
3. For widths 8/16/32 (at least one is mandatory; better all):
  - write TX word
  - expect RX word equals TX word (consider bit order!)

## Checks

- RX==TX in loopback mode (R19).
- External MISO ignored.

## Covers mainly

- R19, and indirectly R6/R7.
- 

## 9) delay\_transfer\_test

**Purpose:** verify idle half-cycles inserted between transfers, BUSY stays 1.

## Stimulus

1. Set DELAY = 0: send two back-to-back TX words; record gap between transfers (should be minimal).
2. Set DELAY = 1: send two words; measure inserted idle half-cycle count.
3. Set DELAY to large ( $\geq 128$ ): send two words; measure long idle duration.
4. Important: queue 2+ words in TX FIFO so “another word is queued” condition is true.

## Checks

- SCLK holds idle level during delay.
- BUSY stays 1 across the delay (per spec).
- Delay value change during BUSY should only take effect next transfer (good extra check).

## Covers mainly

- R21 (and supports R4, R7, R25).
- 

## 10) error\_injection\_test



**Purpose:** provoke “defined errors” and undefined/illegal sequences; check correct handling.

## Stimulus (must include all listed)

### 1. TX write when full

- fill TX FIFO to full
- attempt one more TX\_DATA write
- expect discard + TX\_OVF set (R13)

### 2. RX read when empty

- ensure RX\_EMPTY=1
- read RX\_DATA
- expect return 0 and no RX\_OVF (R15)

### 3. Illegal width encoding (WIDTH=2'b11)

- write CTRL.WIDTH=3 (undefined behavior)
- The spec says undefined, so you **cannot** demand functional output.
- What you **can** do: ensure TB doesn't crash; log that illegal sequence was attempted; optionally assert that reserved bits read as 0 etc.

### 4. Reserved offsets

- read from 0x24+ => must be 0
- write to 0x24+ => ignored (R23)

### 5. (Optional but strong) change MODE/WIDTH/CLK\_DIV while BUSY=1 (illegal)

- again undefined behavior, but can be used to catch common buggy implementations if you add “should not affect current transfer” as an expectation only where spec guarantees (R25).

## Checks

- R13, R15, R23 strongly.
  - If you include RX overflow by filling RX then completing another transfer: that's R14 too.
- 

## 3) Neat summary for your testbench team (copy/paste)

### Mandatory tests — targets & must-hit requirements

#### 1. sanity\_test

- Target: basic config + 1 transfer works
- Req: R7 (basic), R9/R10 (basic), R4(mode0 idle), R20(SS held)

#### 2. reg\_access\_test

- Target: reset + register read/write correctness; APB always-ready/no-error

- Req: R1, R2, R22 (+ optionally R23)

### 3. **mode\_coverage\_test**

- Target: mode correctness (edges/idle), bit ordering, widths across all combos
- Req: R4, R5, R6, R7

### 4. **width\_coverage\_test**

- Target: exact WIDTH cycles + BUSY timing + boundary patterns
- Req: R7, R25 (strong), supports R6

### 5. **fifo\_stress\_test**

- Target: depth=8, full/empty flags, FIFO ordering, back-to-back transfers
- Req: R9, R10, R11, R12 (+ supports R13/R14/R15)

### 6. **interrupt\_test**

- Target: 5 interrupts: assert/mask/W1C/race + IRQ equation
- Req: R16, R17, R18 (+ supports R13/R14)

### 7. **clk\_div\_corner\_test**

- Target: SCLK frequency formula + DIV corners including DIV=0
- Req: R8, R24 (+ supports R25)

### 8. **loopback\_test**

- Target: loopback ignores MISO, RX==TX
- Req: R19 (+ supports R6/R7)

### 9. **delay\_transfer\_test**

- Target: DELAY idle half-cycles inserted; BUSY stays 1 across delay
- Req: R21 (+ supports R4/R7/R25)

### 10. **error\_injection\_test**

- Target: TX overflow, RX empty read, reserved offsets, illegal configs
- Req: R13, R15, R23 (+ optionally R14)